OmniTranslate

中文文档

> Real-time screen OCR and translation tool for Windows.

OmniTranslate captures any region on your screen, continuously extracts text via PaddleOCR, and translates it in real-time using a streaming LLM.

Features

- **Region Selection** — draw a rectangle on screen to define the capture area
- **Real-time OCR** — continuous text extraction using PaddleOCR PP-OCRv4/v3 (offline, 10+ languages)
- **15 Translation Engines** — free Google / Bing, DeepL, Azure, Baidu, Youdao, Tencent, Caiyun, and AI models (DeepSeek, OpenAI, Gemini, Qwen, Kimi, GLM, any OpenAI-compatible API), picked on the *Engine* tab
- **Two Modes**:
  - *Reading Mode* — one-shot translation, best for static text
  - *Real-time Mode* — context-aware incremental translation, best for live subtitles
- **Language Selection** — choose source and target languages from 10+ options
- **Copy to Clipboard** — one-click copy for both OCR and translated text
- **Adjustable Interval** — control OCR frequency (500ms–3000ms)
- **Global Hotkey** — `Ctrl+Shift+S` to trigger region selection from anywhere
- **Debug Monitor** — hidden pipeline inspector for OCR/LLM/display diagnostics (press `F12`)
- **Dark Theme** — Catppuccin Mocha aesthetic

Requirements

- Windows 10 / 11
- uv (`winget install --id=astral-sh.uv -e`) — it downloads Python 3.13 itself, no separate Python install needed
- Internet connection (for translation; OCR works fully offline)
- Optional: an API key for DeepL / DeepSeek / OpenAI / another OpenAI-compatible service

Installation

```
git clone https://github.com/Edolance/OmniTranslate.git
cd OmniTranslate

# Creates .venv\ inside the project folder and installs the locked dependencies
setup.bat
```

Dependencies are declared in `pyproject.toml` and pinned in `uv.lock`, so every install gets exactly the same versions. All scripts use paths relative to the project folder, so it can be cloned or moved anywhere. To set it up by hand:

```
uv venv --relocatable --managed-python
uv sync --locked
```

Optionally pre-download OCR models (otherwise they download on first use):

```
uv run scripts\download_models.py
```

Translation Engine

Open the **Engine** tab and tick one engine; its settings appear on the right:

Group	Engine	Needs
Free (no key)	Google Translate	nothing (may be rate-limited on some networks)
	Microsoft Bing	nothing (≤ 1000 characters per request)
Translation APIs	DeepL	API key (free `:fx` or Pro)
	Microsoft Azure	key, optional region
	Baidu Translate	APP ID + Secret Key
	Youdao (NetEase)	App Key + App Secret
	Tencent TMT	SecretId + SecretKey, optional region
	Caiyun	token
AI models	DeepSeek, OpenAI, Google Gemini, Qwen, Kimi, Zhipu GLM	API key, optional model
	Custom	API key, Base URL, model (any OpenAI-compatible API)

Keys are saved per user in the Windows registry (`HKCU\Software\OmniTranslate`), not in the app folder, so sharing the folder never shares your keys. Real-time context (keeping the previous translation stable) only works with the LLM engines; Google and DeepL re-translate the full text each time.

A `.env` file (see `.env.example`) is still read on first launch and imported into these settings, but it is no longer required.

Usage

```
run.bat
```

or:

```
uv run run.py
```

Standalone exe

`build_exe.bat` packages the app with PyInstaller into `dist\OmniTranslate\` (no Python needed on the target PC):

```
build_exe.bat
```

Copy the whole `dist\OmniTranslate\` folder anywhere, double-click `OmniTranslate.exe` , and pick an engine on the Engine tab. The downloaded OCR models are bundled in `.paddleocr\` ; output is logged to `omnitranslate.log` in the same folder.

To check a build without the GUI, run `OmniTranslate.exe --selftest` : it runs OCR once on a generated image and writes the result to the log.

Workflow

1. Click **Select Region** (or press `Ctrl+Shift+S`)
2. Drag a rectangle over the text you want to capture
3. Choose **Source** and **Target** languages
4. Select mode: **Reading** (static text) or **Real-time** (live subtitles)
5. Click **Start** to begin real-time OCR + translation
6. Use **Copy** buttons to copy text to clipboard
7. Click **Stop** when done

Project Structure

```
omnitranslate/
├── app.py                # Application entry point, global hotkey
├── capture/
│   ├── selector.py       # Transparent overlay for region selection
│   └── grabber.py        # mss-based screen capture (per-call context)
├── ocr/
│   └── engine.py         # PaddleOCR PP-OCRv4/v3 with line grouping
├── translator/
│   └── engine.py         # OpenAI-compatible streaming LLM translation
└── ui/
    └── main_window.py    # Main window, workers, flicker suppression logic
```

Architecture

Package Responsibilities

Package	Role
`omnitranslate.capture`	Screen region selection (`RegionSelector`) and pixel capture (`ScreenGrabber`)
`omnitranslate.ocr`	Text extraction from images via PaddleOCR (`OCREngine`)
`omnitranslate.translator`	Streaming LLM translation with context preservation (`TranslationEngine`)
`omnitranslate.ui`	PyQt5 GUI, multi-threaded pipeline, flicker suppression (`MainWindow`)
`omnitranslate.app`	QApplication bootstrap and global hotkey registration

Data Flow

```
User selects region
       ↓
  mss captures region (RGB array)   ← ThreadPoolExecutor
       ↓
  PaddleOCR extracts text           ← ThreadPoolExecutor
       ↓
  Text changed?  ── No → skip
       ↓ Yes
  Backpressure + throttle gate
       ↓
  LLM streaming translation         ← QThread
       ↓
  Three-level merge into display:
    1. Perfect prefix → append tail
    2. Overlap found  → targeted edit
    3. No overlap     → append paragraph
       ↓
  Auto-scroll to latest content
```

Threading Model

Thread	Component	Reason
Qt Main Thread	UI rendering, signal dispatch	Qt requires GUI on main thread
`ThreadPoolExecutor`	OCR (ScreenGrabber + OCREngine)	PaddlePaddle segfaults in QThread
`QThread`	Translation (LLM API)	Network I/O must not block the UI

Internal Dependencies

```
app.py → ui.main_window → capture.selector
                        → capture.grabber
                        → ocr.engine
                        → translator.engine
```

The `capture`, `ocr`, and `translator` packages are fully independent of each other. Only `ui.main_window` imports from all three.

Technology Stack

Component	Library	Purpose
GUI	PyQt5	Window, overlay, widgets, threading
Screen Capture	mss	Fast screenshot without disk I/O
OCR	PaddleOCR (PaddlePaddle)	Offline OCR — PP-OCRv4 (zh/en), PP-OCRv5 (th), PP-OCRv3 (others)
Translation	openai (Python SDK)	Streaming LLM via any OpenAI-compatible API
Hotkey	pynput	Global keyboard listener
HTTP	requests	Google / DeepL translation calls
Env Config	python-dotenv	Optional legacy `.env` import

Configuration

All runtime settings are controlled via the UI:

Setting	Control	Range / Options
OCR interval	Slider	500ms – 3000ms
Source language	Dropdown	Auto-detect + 10+ languages
Target language	Dropdown	10+ languages
Translation mode	Radio buttons	Reading / Real-time
Window position	Drag	Always on top

License

This project is for personal/educational use.